

Unit-6: Memory Management

Main Topic	Subtopics	Focus
Memory Management	Definition, role of OS, memory allocation	Basic concept
Memory Management Requirements	Relocation, protection, sharing, logical organization	Memory control
Memory Partitioning	Fixed partitioning, variable partitioning	Memory division
Memory Allocation Algorithms	First Fit, Best Fit, Worst Fit	Allocation strategies
Paging	Pages, frames, page table	Non-contiguous allocation
Fragmentation	Internal fragmentation, external fragmentation	Memory wastage
Swapping	Process swapping between RAM and disk	Memory optimization
Demand Paging	Page fault, page loading	Virtual memory concept
Memory Protection & Security	Protection mechanisms	Prevent illegal access

Memory Management

1. Definition : Memory Management is the function of an operating system that **controls and coordinates the use of main memory (RAM)** among multiple processes.

It ensures that each process gets the required memory space while preventing interference between processes.

Simple exam definition : Memory management is the process of allocating, managing, and controlling main memory so that multiple programs can execute efficiently.

Need for Memory Management

In a multiprogramming system, many processes run simultaneously. Since **main memory is limited**, the operating system must manage memory efficiently.

Reasons for memory management:

Reason	Explanation
Efficient memory usage	Prevent memory wastage
Process isolation	Prevent processes from interfering
Multiprogramming support	Allow multiple programs to run
System stability	Prevent illegal memory access

Role of Operating System in Memory Management

The operating system performs several important tasks related to memory.

Function	Description
Memory allocation	Assign memory to processes
Memory deallocation	Release memory after process completion
Memory protection	Prevent unauthorized access
Memory sharing	Allow controlled sharing between processes
Memory tracking	Keep record of used and free memory

Memory Management Structure

Operating System



Memory Manager



Process P1 → Memory

Process P2 → Memory

Process P3 → Memory



RAM

The memory manager allocates and controls memory usage.

Memory Allocation

Memory allocation refers to **assigning memory space to processes**.

Two main types of allocation exist.

Type	Description
Contiguous allocation	Process occupies continuous memory
Non-contiguous allocation	Process stored in multiple locations

Examples:

Technique	Type
Fixed Partitioning	Contiguous
Variable Partitioning	Contiguous
Paging	Non-contiguous
Segmentation	Non-contiguous

Memory Hierarchy

Computers use different types of memory with varying speeds.

Memory Type	Speed	Example
Registers	Fastest	CPU registers
Cache	Very fast	CPU cache
Main Memory	Moderate	RAM
Secondary Memory	Slow	Hard disk, SSD

The operating system mainly manages **main memory (RAM)**.

Example of Memory Allocation

Suppose the system has **1024 MB RAM**.

Process	Memory Required
P1	200 MB
P2	300 MB
P3	150 MB

The OS allocates memory to each process and manages the remaining memory.

Advantages of Memory Management

Advantage	Explanation
Efficient memory utilization	Better system performance
Supports multitasking	Multiple processes run simultaneously
Prevents memory conflicts	Processes remain isolated

Problems Without Memory Management

Without proper memory management:

Problem	Explanation
Memory conflicts	Processes overwrite each other's data
System crashes	Incorrect memory access

Poor performance	Inefficient memory usage
------------------	--------------------------

Key Points for Exams

- Memory management controls **main memory allocation**
- OS tracks **used and free memory**
- Supports **multiprogramming**
- Provides **memory protection and sharing**

Short Conclusion

Memory management is a fundamental function of the operating system that allocates and controls main memory for different processes. It ensures efficient utilization of memory, prevents conflicts between processes, and supports stable execution of programs in a multitasking environment.

Memory Management Requirements

1. Definition : Memory Management Requirements are the conditions or features that an operating system must provide to manage memory efficiently and safely when multiple processes are running. These requirements ensure that processes can execute correctly without interfering with each other.

Simple exam definition : Memory management requirements are the essential conditions that ensure efficient and secure allocation and use of memory by processes.

Main Requirements of Memory Management

There are four major requirements.

Requirement	Purpose
Relocation	Allows processes to be moved in memory
Protection	Prevents processes from accessing other processes' memory
Sharing	Allows controlled sharing of memory between processes
Logical Organization	Organizes programs into logical units

1. Relocation

Concept : Relocation means that a process can be **loaded or moved to any location in memory during execution**.

This is necessary because memory allocation changes when multiple processes run simultaneously.

Example : Suppose a program is originally designed to run at memory address **1000**.

If memory location **1000 is already occupied**, the program must be loaded at another address such as **5000**.

Original Address → 1000

Loaded Address → 5000

The operating system adjusts the addresses accordingly.

Importance

Benefit	Explanation
Flexible memory allocation	Programs can run anywhere in memory
Efficient memory usage	OS can rearrange processes

2. Protection

Concept : Memory protection ensures that a process **cannot access or modify memory belonging to another process**.

Each process must operate within its allocated memory space.

Example

If Process P1 tries to access memory assigned to Process P2, the operating system blocks the action.

Process P1 → Memory P1 (Allowed)

Process P1 → Memory P2 (Not Allowed)

Importance

Benefit	Explanation
Prevents program errors	Processes cannot corrupt each other
Improves system security	Protects sensitive data

3. Sharing

Concept : Memory sharing allows multiple processes to **share common data or programs** while maintaining protection.

This improves system efficiency.

Example

In many systems, multiple users may run the same program (such as a text editor).

Instead of loading the program multiple times, the operating system **shares the same code in memory**.

Shared Program

↓

Process P1

Process P2

Process P3

Importance

Benefit	Explanation
Reduces memory usage	Shared code stored once
Improves efficiency	Faster program execution

4. Logical Organization

Concept

Programs are usually divided into **logical modules** such as:

- main program
- functions
- data structures

Logical organization helps manage these modules efficiently.

Example

Program

├— Code Section

├— Data Section

├— Stack

└— Heap

Each part of the program serves a specific purpose.

Importance

Benefit	Explanation
Better program structure	Organized program modules
Easier memory management	Separate memory areas

Summary of Memory Management Requirements

Requirement	Main Purpose
Relocation	Move processes in memory
Protection	Prevent illegal memory access
Sharing	Allow multiple processes to share resources
Logical Organization	Organize programs efficiently

Key Points for Exams

- Memory management must satisfy **four requirements**
- These ensure **efficient and secure memory usage**
- Important concepts:
 - Relocation
 - Protection
 - Sharing
 - Logical organization

Short Conclusion

Memory management requirements define the essential conditions needed for efficient and secure memory usage in a computer system. By providing relocation, protection, sharing, and logical organization, the operating system ensures that multiple processes can execute safely and efficiently in memory.

Memory Partitioning

1. Definition : Memory Partitioning is a memory management technique in which **main memory (RAM) is divided into several sections called partitions**, and each partition is allocated to a process.

This method allows multiple processes to **reside in memory simultaneously**.

Simple exam definition : Memory partitioning is the technique of dividing main memory into parts so that multiple processes can be loaded and executed.

Purpose of Memory Partitioning

Memory partitioning helps the operating system manage memory efficiently.

Purpose	Explanation
Support multiprogramming	Multiple processes run at the same time
Efficient memory usage	Memory is divided into manageable sections
Better process management	OS allocates partitions to processes

Types of Memory Partitioning

There are two main types of memory partitioning.

Type	Description
Fixed Partitioning	Memory is divided into fixed-size partitions
Variable Partitioning	Memory partitions are created dynamically

1. Fixed Partitioning

Concept : In **fixed partitioning**, memory is divided into **fixed-size partitions before program execution**.

Each partition can hold **one process at a time**.

Example:

Main Memory

Partition 1 → Process P1

Partition 2 → Process P2

Partition 3 → Process P3

Partition 4 → Free

Characteristics

Feature	Description
Partitions are fixed	Size determined in advance
One process per partition	Only one process allowed
Simple implementation	Easy memory management

Advantages

Advantage	Explanation
-----------	-------------

Simple to implement	Easy memory allocation
Fast allocation	OS quickly assigns partitions

Disadvantages

Disadvantage	Explanation
Internal fragmentation	Unused memory inside partition
Limited number of processes	Number of partitions fixed

2. Variable Partitioning

Concept : In **variable partitioning**, memory is divided into **different-sized partitions based on process requirements**.

The operating system dynamically allocates memory to processes.

Example:

Main Memory

Process P1 → 120 MB

Process P2 → 300 MB

Process P3 → 150 MB

Free Memory

Characteristics

Feature	Description
Dynamic partitions	Created according to process size
Flexible allocation	Memory allocated as needed
Efficient usage	Less internal fragmentation

Advantages

Advantage	Explanation
Better memory utilization	Partitions fit process size
Flexible allocation	Supports processes of different sizes

Disadvantages

Disadvantage	Explanation
External fragmentation	Free memory scattered
Complex management	Harder for OS to manage

Comparison of Fixed and Variable Partitioning

Feature	Fixed Partitioning	Variable Partitioning
Partition size	Fixed	Dynamic
Memory utilization	Lower	Better
Fragmentation	Internal fragmentation	External fragmentation
Implementation	Simple	Complex

Simple Partitioning Diagram

Main Memory

Operating System

Process P1

Process P2

Free Space

The OS allocates partitions to processes as needed.

Key Points for Exams

- Memory partitioning divides **RAM into partitions**
- Two types:
 - Fixed partitioning
 - Variable partitioning
- Fixed partitioning causes **internal fragmentation**
- Variable partitioning may cause **external fragmentation**

Short Conclusion

Memory partitioning is a technique used by operating systems to divide main memory into partitions for process execution. Fixed partitioning uses predefined memory blocks, while variable partitioning dynamically allocates memory based on process requirements.

Fixed Partitioning

1. Definition: Fixed Partitioning is a memory management technique in which **main memory is divided into a fixed number of partitions before system execution.**

Each partition has a **fixed size**, and **only one process can occupy a partition at a time.**

Simple exam definition: Fixed partitioning is a memory allocation method where memory is divided into fixed-size partitions and each partition holds one process.

Basic Concept : In fixed partitioning, the operating system divides the available memory into **several fixed sections.**

Example:

Main Memory

Operating System

Partition 1 → Process P1

Partition 2 → Process P2

Partition 3 → Process P3

Partition 4 → Free

Each partition is allocated to a single process.

Characteristics of Fixed Partitioning

Feature	Description
Predefined partitions	Memory divided before execution
One process per partition	Each partition holds one process
Simple implementation	Easy to manage
Limited multiprogramming	Number of processes limited

Types of Fixed Partitioning

There are two types of fixed partitioning.

Type	Description
Equal-size partitions	All partitions have same size
Unequal-size partitions	Partitions have different sizes

1. Equal-Size Partitioning

In this method, **all partitions have the same size.**

Example:

Memory

OS

Partition (200 MB)

Partition (200 MB)

Partition (200 MB)

Partition (200 MB)

Small processes may waste memory space.

2. Unequal-Size Partitioning

In this method, partitions have **different sizes**.

Example:

Memory

OS

Partition (100 MB)

Partition (200 MB)

Partition (300 MB)

Partition (400 MB)

Processes are placed in partitions that best match their size.

Example

Suppose memory has **4 partitions**:

Partition	Size
P1	200 MB
P2	200 MB
P3	200 MB
P4	200 MB

Processes:

Process	Memory Required
A	150 MB
B	120 MB
C	180 MB

Each process occupies one partition.

Problem: Internal Fragmentation

In fixed partitioning, some memory space may remain unused inside a partition.

Example:

Partition Size	Process Size	Wasted Memory
200 MB	150 MB	50 MB

This unused memory is called **internal fragmentation**.

Advantages of Fixed Partitioning

Advantage	Explanation
Simple to implement	Easy memory management
Fast allocation	Quick process loading
Low overhead	Minimal OS complexity

Disadvantages of Fixed Partitioning

Disadvantage	Explanation
Internal fragmentation	Unused memory inside partitions
Limited number of processes	Fixed number of partitions
Inefficient memory use	Large partitions may remain partially empty

Fixed Partitioning Diagram

Main Memory

Operating System

Partition 1 → Process A

Partition 2 → Process B

Partition 3 → Process C

Partition 4 → Free

Key Points for Exams

- Memory divided into **fixed partitions**
- One process per partition
- Two types:
 - Equal-size partitions
 - Unequal-size partitions
- Main problem: **Internal fragmentation**

Short Conclusion

Fixed partitioning is a simple memory management technique where main memory is divided into fixed-size partitions before execution. Each partition can hold one process, but this method may lead to internal fragmentation and inefficient memory utilization.

Variable Partitioning

1. Definition : Variable Partitioning is a memory management technique in which **main memory is divided into partitions of different sizes based on the requirements of processes.**

Unlike fixed partitioning, the partitions are **not predefined**. Instead, they are created **dynamically when processes are loaded into memory.**

Simple exam definition : Variable partitioning is a memory allocation method where memory partitions are created dynamically according to the size of processes.

Basic Concept

In variable partitioning, memory is allocated exactly according to the **size of the process.**

Example:

Main Memory

Operating System

Process P1 → 120 MB

Process P2 → 200 MB

Process P3 → 150 MB

Free Memory

The operating system allocates memory depending on process requirements.

Working of Variable Partitioning

Steps involved:

1. A process arrives in the system.
2. The operating system searches for **free memory space**.
3. A partition is created according to the **size of the process**.
4. The process is loaded into that partition.

Example

Suppose total memory = **1000 MB**

Processes require:

Process	Memory Required
P1	200 MB
P2	300 MB
P3	150 MB

Memory allocation:

Main Memory

OS

P1 → 200 MB

P2 → 300 MB

P3 → 150 MB

Free → 350 MB

Partitions are created based on process sizes.

Problem: External Fragmentation

In variable partitioning, free memory may become **scattered into small pieces**.

Example:

Memory

P1

Free Space

P2

Free Space

P3

Free Space

Although total free memory may be sufficient, it may **not be contiguous**, causing allocation problems.

This problem is called **external fragmentation**.

Solution: Compaction

Compaction is used to solve external fragmentation.

The operating system **rearranges processes in memory** so that free memory becomes one continuous block.

Example:

Before Compaction

P1

Free

P2

Free

P3

After Compaction

P1

P2

P3

Free Memory

Advantages of Variable Partitioning

Advantage	Explanation
Better memory utilization	Memory allocated according to process size
Less internal fragmentation	Minimal unused memory inside partitions
Flexible allocation	Supports processes of different sizes

Disadvantages of Variable Partitioning

Disadvantage	Explanation
External fragmentation	Free memory becomes scattered
Complex management	OS must track memory continuously
Compaction overhead	Moving processes takes time

Variable Partitioning Diagram

Main Memory

Operating System

Process P1 (120 MB)

Process P2 (300 MB)

Process P3 (150 MB)

Free Space

Comparison with Fixed Partitioning

Feature	Fixed Partitioning	Variable Partitioning
Partition size	Fixed	Dynamic
Memory utilization	Lower	Better
Fragmentation	Internal fragmentation	External fragmentation
Flexibility	Less flexible	More flexible

Key Points for Exams

- Partitions are created **dynamically**
- Memory allocated according to **process size**
- Main problem: **External fragmentation**
- Solution: **Compaction**

Short Conclusion

Variable partitioning is a dynamic memory allocation technique where memory partitions are created based on process requirements. It improves memory utilization compared to fixed partitioning but may lead to external fragmentation, which can be solved using compaction.

Address Binding

1. Introduction

When a program is written, it uses addresses to access instructions and data.

However:

- actual physical memory location may change during execution
- operating system must map program addresses to real memory addresses

This process is called:

Address Binding

Definition: Address Binding is the process of mapping program-generated addresses to actual memory addresses.

Simple Exam Definition: Address binding is the process of connecting logical addresses of a program to physical memory addresses.

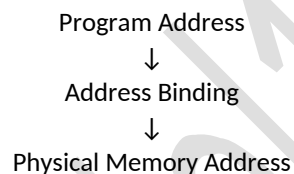
Why Address Binding is Needed

Programs are created without knowing:

- exact memory location
- future memory availability

The OS binds addresses dynamically during loading or execution.

Basic Idea



Types of Address Binding

There are three types of address binding.

Type	Binding Time
Compile-Time Binding	During compilation
Load-Time Binding	During loading
Execution-Time Binding	During execution

1. Compile-Time Binding

Concept

Binding occurs during program compilation.

Compiler generates:
absolute addresses

Working

If memory location is known in advance:

- compiler directly generates physical addresses

Example

Suppose program always loaded at:

Address 1000

Compiler creates addresses accordingly.

Characteristics

Feature	Explanation
Fixed memory location	Predefined
No relocation possible	Cannot move program
Fast execution	No runtime mapping

Advantages

Advantage	Explanation
Simple implementation	Easy binding
Faster execution	Direct addresses

Disadvantages

Disadvantage	Explanation
Inflexible	Program cannot move
Recompilation needed	If location changes

2. Load-Time Binding

Concept: Binding occurs when program is loaded into memory.

Working

Compiler generates:

relocatable addresses

Loader converts them into physical addresses.

Example

Program can load at:

1000

or

5000

or

8000

Loader adjusts addresses.

Characteristics

Feature	Explanation
Flexible loading	Program movable before execution
Relocation possible	During loading
Moderate complexity	Loader handles mapping

Advantages

Advantage	Explanation
Better flexibility	Program movable
Efficient memory usage	Dynamic loading

Disadvantages

Disadvantage	Explanation
Cannot move during execution	Fixed after loading

3. Execution-Time Binding

Concept

Binding occurs while program is executing.

Also called: Dynamic Binding

Working

Logical addresses generated continuously during execution.

Hardware converts them into physical addresses.

Uses MMU

Execution-time binding uses: Memory Management Unit (MMU)

Example: Program may move dynamically between memory locations during execution.

Characteristics

Feature	Explanation
Maximum flexibility	Dynamic relocation
Supports virtual memory	Modern systems
Hardware support needed	MMU required

Advantages

Advantage	Explanation
Dynamic relocation	Process movable anytime
Better memory utilization	Flexible allocation
Supports multiprogramming	Efficient execution

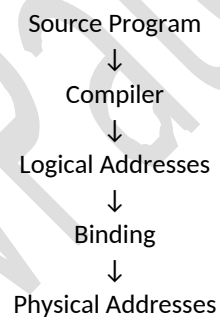
Disadvantages

Disadvantage	Explanation
Complex implementation	Hardware required
Additional overhead	Address translation time

Comparison of Address Binding Types

Feature	Compile-Time	Load-Time	Execution-Time
Binding occurs	Compilation	Loading	Execution
Relocation	Not possible	Before execution	During execution
Flexibility	Low	Medium	High
Hardware support	Not needed	Not needed	Required

Address Binding Diagram



Real-Life Analogy

Hotel Room Allocation

Situation	Binding Type
Fixed room reserved forever	Compile-time
Room assigned at check-in	Load-time
Room changed during stay	Execution-time

Importance of Address Binding

Importance	Explanation
Efficient memory use	Flexible allocation
Process relocation	Dynamic execution
Multiprogramming support	Many processes
Virtual memory support	Modern OS systems

Key Terms

Term	Meaning
------	---------

Logical Address	Program-generated address
Physical Address	Actual RAM location
Relocation	Moving process in memory
MMU	Memory Management Unit

Important Exam Points

- Address binding maps logical to physical addresses
- Three types:
 - Compile-time
 - Load-time
 - Execution-time
- Execution-time binding uses MMU
- Dynamic binding provides maximum flexibility

Short Conclusion

Address Binding is the process of mapping program-generated logical addresses to actual physical memory addresses. It can occur during compilation, loading, or execution. Modern operating systems mainly use execution-time binding because it supports dynamic relocation, virtual memory, and efficient multiprogramming.

Logical and Physical Address

1. Introduction: When a program executes, it generates addresses to access instructions and data. However:

- these addresses are not the actual RAM addresses
- operating system converts them into real physical addresses

Thus two types of addresses exist:

- Logical Address
- Physical Address

Definition of Logical Address : A Logical Address is the address generated by the CPU during program execution.

Also called: Virtual Address

Simple Exam Definition : Logical address is the address generated by the CPU for a process.

Definition of Physical Address: A Physical Address is the actual address present in main memory (RAM).

Simple Exam Definition: Physical address is the real memory location in RAM.

Basic Idea

CPU → Logical Address



MMU



Physical Address → RAM

Logical Address Space

The set of all logical addresses generated by a program.

Physical Address Space

The set of all physical addresses available in RAM.

Example

Suppose:

Logical Address = 150

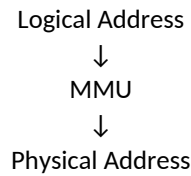
MMU converts it into:

Physical Address = 4150

Memory Management Unit (MMU)

The hardware responsible for address translation.

Working of MMU



Address Translation

The process of converting logical addresses into physical addresses.

Example of Address Translation

Suppose:

- Base register = 4000
- Logical address = 200

Then:

$$\text{Physical Address} = \text{Base Address} + \text{Logical Address}$$

Result:

Physical Address = 4200

Characteristics of Logical Address

Feature	Explanation
Generated by CPU	During execution
Virtual in nature	Not actual RAM
Used by process	Process view of memory

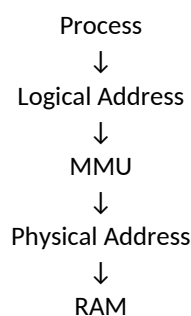
Characteristics of Physical Address

Feature	Explanation
Actual RAM address	Real location
Used by memory hardware	Hardware access
Generated after translation	By MMU

Logical vs Physical Address

Feature	Logical Address	Physical Address
Generated by	CPU	MMU
Visibility	Visible to process	Visible to memory unit
Nature	Virtual	Real
Address space	Logical address space	Physical address space

Diagram of Address Translation



Importance of Logical Address

Importance	Explanation
Process isolation	Processes separated
Virtual memory support	Large logical space
Security	Indirect memory access

Importance of Physical Address

Importance	Explanation
Actual memory access	RAM communication
Hardware operation	Physical storage

Advantages of Logical Addressing

Advantage	Explanation
Process protection	Isolation between processes
Flexible memory management	Dynamic allocation
Supports virtual memory	Larger address space

Disadvantages

Disadvantage	Explanation
Translation overhead	Extra processing
Hardware dependency	MMU needed

Real-Life Analogy

House Address Example

Concept	Example
Logical Address	Person's requested location
Physical Address	Actual house location

Logical Address Space vs Physical Address Space

Feature	Logical Space	Physical Space
Managed by	CPU	Memory hardware
Size	Can be larger	Limited by RAM
Usage	Program execution	Actual storage

Important Exam Points

- Logical address generated by CPU
- Physical address exists in RAM
- MMU performs translation
- Logical address also called virtual address
- Address translation important for virtual memory

Key Terms

Term	Meaning
Logical Address	CPU-generated address
Physical Address	Actual RAM location
MMU	Address translation hardware
Virtual Address	Another name for logical address

Short Conclusion

Logical addresses are virtual addresses generated by the CPU during program execution, while physical addresses are the actual locations in main memory. The Memory Management Unit (MMU) converts logical

addresses into physical addresses through address translation. This mechanism enables process protection, virtual memory, and efficient memory management in modern operating systems.

Memory Allocation

1. Definition : Memory Allocation is the process by which the operating system **assigns available memory space to processes** so that they can execute.

Since multiple processes run in the system, the operating system must decide **which memory block should be allocated to which process**.

Simple exam definition

Memory allocation is the technique used by the operating system to assign memory blocks to processes.

Purpose of Memory Allocation

Purpose	Explanation
Efficient memory use	Utilize available memory properly
Process execution	Provide memory space for programs
Support multiprogramming	Allow multiple processes in memory
Reduce fragmentation	Improve memory utilization

Memory Allocation Algorithms : When free memory blocks are available, the operating system must choose a suitable block for a process. This decision is made using **memory allocation algorithms**.

The three main algorithms are:

Algorithm	Description
First Fit	Allocate the first available memory block
Best Fit	Allocate the smallest block that fits the process
Worst Fit	Allocate the largest available block

1. First Fit Algorithm

Concept : The **First Fit** algorithm allocates the **first free memory block that is large enough to satisfy the process requirement**.

The search starts from the beginning of memory.

Example

Free memory blocks:

Block	Size
B1	100 KB
B2	500 KB
B3	200 KB
B4	300 KB

Process size = **150 KB**

Allocation using **First Fit**:

- B1 → too small
- B2 → allocated

Diagram

Memory Blocks

100 KB
500 KB ← Allocated
200 KB
300 KB

Advantages

Advantage	Explanation
Fast allocation	Stops at first suitable block
Simple implementation	Easy algorithm

Disadvantages

Disadvantage	Explanation
External fragmentation	Free memory becomes scattered
May waste large blocks	Large blocks used early

2. Best Fit Algorithm

Concept : The **Best Fit** algorithm allocates the **smallest free memory block that is large enough for the process.**

This reduces wasted memory.

Example

Free blocks:

Block	Size
100 KB	
500 KB	
200 KB	
300 KB	

Process size = **150 KB**

Allocation using **Best Fit**:

- 200 KB block is chosen (smallest sufficient block)

Diagram

Memory Blocks

 100 KB
 500 KB
 200 KB ← Allocated
 300 KB

Advantages

Advantage	Explanation
Better memory utilization	Less leftover space
Efficient allocation	Smaller blocks used

Disadvantages

Disadvantage	Explanation
Slow allocation	Must search entire memory
External fragmentation	Many small unused blocks

3. Worst Fit Algorithm

Concept: The **Worst Fit** algorithm allocates the **largest available memory block** to the process.

The idea is to leave large leftover space for future processes.

Example

Free blocks:

Block	Size
100 KB	
500 KB	

200 KB	
300 KB	

Process size = **150 KB**

Allocation using **Worst Fit**:

- 500 KB block is selected.

Diagram

Memory Blocks

 100 KB
 500 KB ← Allocated
 200 KB
 300 KB

Comparison of Memory Allocation Algorithms

Feature	First Fit	Best Fit	Worst Fit
Search speed	Fast	Slow	Moderate
Memory utilization	Moderate	Better	Poor
Fragmentation	Moderate	High	Moderate

Example Summary

Suppose memory blocks:

Block	Size
100 KB	
500 KB	
200 KB	
300 KB	

Process size = **150 KB**

Algorithm	Allocated Block
First Fit	500 KB
Best Fit	200 KB
Worst Fit	500 KB

Key Points for Exams

- Memory allocation assigns **memory blocks to processes**
- Three main algorithms:
 - First Fit
 - Best Fit
 - Worst Fit
- These algorithms help **select suitable memory blocks**

Short Conclusion

Memory allocation algorithms are used by operating systems to decide how free memory blocks are assigned to processes. Techniques such as First Fit, Best Fit, and Worst Fit help manage memory efficiently while supporting multiprogramming systems.

First Fit Algorithm

1. Definition : First Fit is a memory allocation algorithm used in operating systems where the **first available memory block that is large enough to accommodate a process is allocated**.

The operating system scans memory from the **beginning and assigns the first suitable block it finds**.

Simple exam definition : First Fit is a memory allocation technique that allocates the first free memory block large enough for a process.

Working of First Fit

Steps involved:

1. The operating system maintains a list of **free memory blocks**.
2. When a process arrives, the OS **searches memory from the beginning**.
3. The **first block that is large enough** for the process is selected.
4. The process is loaded into that memory block.
5. The remaining part of the block becomes **free memory**.

Example

Free memory blocks:

Block	Size
B1	100 KB
B2	500 KB
B3	200 KB
B4	300 KB

Process size = **150 KB**

Allocation process:

Step	Action
B1	Too small
B2	Allocated

Thus the process is allocated in **Block B2**.

First Fit Diagram

Memory Blocks

100 KB
500 KB ← Allocated
200 KB
300 KB

The first block that satisfies the process requirement is selected.

Another Example

Free memory:

Block	Size
100 KB	
500 KB	
200 KB	
300 KB	

Processes:

Process	Size
P1	212 KB
P2	417 KB
P3	112 KB

Allocation using **First Fit**:

Process	Allocated Block
P1	500 KB
P2	Not allocated (no block large enough)
P3	200 KB

Advantages of First Fit

Advantage	Explanation
-----------	-------------

Fast allocation	Stops searching once block found
Simple implementation	Easy algorithm
Low overhead	Minimal searching required

Disadvantages of First Fit

Disadvantage	Explanation
External fragmentation	Free memory becomes scattered
Large blocks may be wasted	Larger blocks used early

Characteristics of First Fit

Feature	Description
Search method	Sequential search
Allocation rule	First suitable block
Speed	Fast

Key Points for Exams

- First Fit allocates **first suitable memory block**
- Search starts **from beginning of memory**
- Faster than other allocation algorithms
- May cause **external fragmentation**

Short Conclusion

The First Fit algorithm is a simple and fast memory allocation technique that assigns the first available memory block large enough for a process. While it provides quick allocation, it may lead to external fragmentation due to scattered free memory blocks.

Best Fit Algorithm

1. Definition : Best Fit is a memory allocation algorithm in which the operating system **allocates the smallest available memory block that is large enough to accommodate the process**. The algorithm searches the **entire list of free memory blocks** and selects the block that leaves the **least unused memory space**.

Simple exam definition : Best Fit is a memory allocation technique that allocates the smallest free block that can hold the process.

Working of Best Fit

Steps involved:

1. The operating system maintains a list of **free memory blocks**.
2. When a process arrives, the OS searches **all free memory blocks**.
3. It selects the **smallest block that is large enough** for the process.
4. The process is loaded into that block.
5. The remaining memory becomes **free space**.

Example

Free memory blocks:

Block	Size
B1	100 KB
B2	500 KB
B3	200 KB
B4	300 KB

Process size = **150 KB**

Allocation using **Best Fit**:

Block	Result
100 KB	Too small
500 KB	Possible
200 KB	Best fit

300 KB	Larger than needed
--------	--------------------

Therefore, **200 KB block is selected.**

Best Fit Diagram

Memory Blocks

 100 KB
 500 KB
 200 KB ← Allocated
 300 KB

The smallest suitable block is chosen.

Example with Multiple Processes

Free memory blocks:

Block	Size
100 KB	
500 KB	
200 KB	
300 KB	

Processes:

Process	Size
P1	212 KB
P2	417 KB
P3	112 KB

Allocation using **Best Fit**:

Process	Allocated Block
P1	300 KB
P2	500 KB
P3	200 KB

Advantages of Best Fit

Advantage	Explanation
Better memory utilization	Uses smallest suitable block
Less wasted space	Minimizes leftover memory

Disadvantages of Best Fit

Disadvantage	Explanation
Slow allocation	Must search entire memory list
External fragmentation	Many small leftover blocks created

Characteristics of Best Fit

Feature	Description
Search method	Full memory search
Allocation rule	Smallest suitable block
Speed	Slower than First Fit

Comparison with First Fit

Feature	First Fit	Best Fit
Search	Stops early	Searches entire memory
Speed	Faster	Slower
Memory utilization	Moderate	Better

Key Points for Exams

- Best Fit allocates **smallest suitable memory block**
- Requires **searching entire free memory list**
- Reduces wasted memory space
- May cause **external fragmentation**

Short Conclusion

The Best Fit algorithm is a memory allocation technique that assigns the smallest possible free memory block to a process. Although it improves memory utilization, it requires searching the entire memory list and may lead to external fragmentation.

Worst Fit Algorithm

1. Definition : Worst Fit is a memory allocation algorithm in which the operating system **allocates the largest available memory block to a process.**

The idea is to leave a **large remaining free block** after allocation so that future processes can be accommodated easily.

Simple exam definition : Worst Fit is a memory allocation technique that allocates the largest available free memory block to a process.

Working of Worst Fit

Steps involved:

1. The operating system maintains a list of **free memory blocks.**
2. When a process arrives, the OS searches **all available blocks.**
3. It selects the **largest memory block.**
4. The process is loaded into that block.
5. The remaining part becomes **free memory.**

Example

Free memory blocks:

Block	Size
B1	100 KB
B2	500 KB
B3	200 KB
B4	300 KB

Process size = **150 KB**

Allocation using **Worst Fit**:

- The largest block is **500 KB.**
- The process is allocated in that block.

Worst Fit Diagram

Memory Blocks

100 KB
500 KB ← Allocated
200 KB
300 KB

The largest available block is selected.

Example with Multiple Processes

Free memory blocks:

Block	Size
100 KB	

500 KB	
200 KB	
300 KB	

Processes:

Process	Size
P1	212 KB
P2	417 KB
P3	112 KB

Allocation using **Worst Fit**:

Process	Allocated Block
P1	500 KB
P2	Not allocated (no block large enough)
P3	300 KB

Advantages of Worst Fit

Advantage	Explanation
Large remaining space	Leaves large blocks for future processes
Simple concept	Easy to understand

Disadvantages of Worst Fit

Disadvantage	Explanation
Poor memory utilization	Large blocks may be wasted
External fragmentation	Memory becomes scattered
Slower search	Must examine all memory blocks

Characteristics of Worst Fit

Feature	Description
Search method	Full memory search
Allocation rule	Largest memory block
Speed	Slower than First Fit

Comparison of Allocation Algorithms

Algorithm	Block Selected	Speed	Memory Usage
First Fit	First suitable block	Fast	Moderate
Best Fit	Smallest suitable block	Slow	Efficient
Worst Fit	Largest block	Slow	Inefficient

Key Points for Exams

- Worst Fit allocates **largest available block**
- Requires **searching entire free memory**
- Leaves large remaining memory space
- May cause **external fragmentation**

Short Conclusion

The Worst Fit algorithm allocates the largest available memory block to a process. Although it leaves large free spaces for future allocations, it may result in poor memory utilization and increased external fragmentation.

Paging

1. Definition : Paging is a memory management technique in which **both the logical memory (process memory) and physical memory are divided into fixed-size blocks.**

- Logical memory blocks are called **Pages**
- Physical memory blocks are called **Frames**

Paging allows a process to be stored in **non-contiguous memory locations**, which helps eliminate **external fragmentation**.

Simple exam definition : Paging is a memory management technique that divides memory into pages and frames and maps pages to frames for efficient memory utilization.

Basic Concept of Paging

In paging:

Memory Type	Block Name
Logical memory	Page
Physical memory	Frame

A page can be stored in **any available frame in RAM**.

Paging Structure

Logical Memory (Process)

Page 0

Page 1

Page 2

Page 3

Physical Memory (RAM)

Frame 0

Frame 1

Frame 2

Frame 3

Pages are loaded into available frames.

Page Table

The **Page Table** is a data structure used by the operating system to **map pages to frames**.

It keeps track of where each page of a process is stored in physical memory.

Example:

Page Number	Frame Number
Page 0	Frame 3
Page 1	Frame 1
Page 2	Frame 4
Page 3	Frame 2

The page table helps the system **translate logical addresses into physical addresses**.

Address Translation in Paging

Logical address consists of two parts.

Part	Meaning
Page number	Identifies the page
Offset	Position within the page

Example:

Logical Address = (Page Number + Offset)

The operating system uses the **page table** to convert this into a **physical address**.

Paging Diagram

Process Pages Physical Memory Frames

Page 0 -----> Frame 2

Page 1 -----> Frame 5

Page 2 -----> Frame 1

Page 3 -----> Frame 4

Pages can be placed anywhere in memory.

Advantages of Paging

Advantage	Explanation
Eliminates external fragmentation	Pages fit into frames easily
Efficient memory use	Non-contiguous allocation
Supports virtual memory	Allows large programs to run

Disadvantages of Paging

Disadvantage	Explanation
Internal fragmentation	Unused space in frames
Page table overhead	Additional memory needed
Address translation time	Extra processing required

Example

Suppose:

- Page size = **4 KB**
- Process size = **16 KB**

Number of pages required:

$16 \text{ KB} / 4 \text{ KB} = 4 \text{ pages}$

These 4 pages will be stored in **4 frames in physical memory**.

Key Points for Exams

- Paging divides memory into **pages and frames**
- Uses **page table** for address mapping
- Eliminates **external fragmentation**
- Allows **non-contiguous memory allocation**

Short Conclusion

Paging is a memory management technique that divides logical and physical memory into fixed-size blocks called pages and frames. It allows efficient memory utilization by enabling processes to be stored in non-contiguous locations while eliminating external fragmentation.

Fragmentation

1. Definition : Fragmentation is a memory management problem in which **available memory becomes divided into small unusable pieces**, reducing the efficiency of memory utilization.

Fragmentation occurs when memory is allocated and deallocated repeatedly while processes are running.

Simple exam definition : Fragmentation is the situation where memory is wasted because it is divided into small unused blocks.

Types of Fragmentation

There are two main types of fragmentation.

Type	Description
Internal Fragmentation	Unused space inside allocated memory block
External Fragmentation	Free memory scattered in small blocks

1. Internal Fragmentation

Concept : Internal Fragmentation occurs when a **process does not fully utilize the allocated memory block**, leaving unused space inside the partition.

This usually happens in **fixed partitioning or paging systems**.

Example

Suppose a partition size is **200 KB**, but the process requires only **150 KB**.

Partition Size	Process Size	Wasted Memory
200 KB	150 KB	50 KB

The unused **50 KB inside the partition** is called internal fragmentation.

Internal Fragmentation Diagram

Memory Partition

Process P1 → 150 KB

Unused Space → 50 KB

The unused space cannot be used by other processes.

2. External Fragmentation

Concept : External Fragmentation occurs when **free memory is divided into many small scattered blocks**.

Although total free memory may be enough, it may **not be contiguous**, making it impossible to allocate memory to a process.

Example

Suppose free memory is:

Free Block	Size
Block 1	100 KB
Block 2	150 KB
Block 3	50 KB

A process requires **200 KB**, but no single contiguous block is large enough.

External Fragmentation Diagram

Memory

Process P1

Free Space

Process P2

Free Space

Process P3

Free Space

Free memory exists but is scattered.

Solution to External Fragmentation

Compaction : Compaction is used to solve external fragmentation.

The operating system **rearranges processes in memory** so that free memory becomes one large block.

Example:

Before Compaction

P1

Free

P2

Free

P3

After Compaction

P1

P2
P3
Free Memory

Comparison of Fragmentation Types

Feature	Internal Fragmentation	External Fragmentation
Location	Inside allocated block	Outside allocated blocks
Cause	Fixed-size allocation	Scattered free memory
Occurs in	Fixed partitioning, paging	Variable partitioning
Solution	Better allocation methods	Compaction

Key Points for Exams

- Fragmentation causes **memory wastage**
- Two types:
 - Internal fragmentation
 - External fragmentation
- External fragmentation can be reduced by **compaction**
- Paging helps reduce **external fragmentation**

Short Conclusion

Fragmentation is a memory management issue where memory becomes inefficiently utilized due to unused or scattered memory spaces. Internal fragmentation occurs inside allocated memory blocks, while external fragmentation occurs when free memory is scattered across multiple small blocks.

Swapping

1. Definition: Swapping is a memory management technique in which a process is **temporarily moved from main memory (RAM) to secondary storage (disk)** and later brought back to memory for execution.

This technique helps the operating system **free memory space and support multiprogramming**.

Simple exam definition: Swapping is the process of transferring a process from main memory to disk and bringing it back into memory when needed.

Purpose of Swapping

Swapping is used when the system has **limited memory but multiple processes need to execute**.

Purpose	Explanation
Free memory space	Move inactive processes to disk
Support multiprogramming	Allow more processes in system
Improve CPU utilization	CPU remains active while processes are swapped

Basic Concept of Swapping

In swapping, a process is moved between **RAM and secondary storage**.

Steps:

1. A process in memory is **swapped out to disk**.
2. Another process is **loaded into RAM**.
3. When required, the swapped-out process is **swapped back into memory**.

Swapping Process Diagram

Main Memory (RAM)	Secondary Storage (Disk)
-----	-----
Process P1	Process P3
Process P2	Process P4
-----	-----

Swap Out → P2 moved to disk

Swap In → P3 loaded into RAM

The operating system manages the swapping process.

Swapping Operations

Two main operations occur during swapping.

Operation	Description
Swap Out	Process moved from RAM to disk
Swap In	Process moved from disk to RAM

Example

Suppose the system has **three memory slots**.

Memory Slot	Process
Slot 1	P1
Slot 2	P2
Slot 3	P3

If a new process **P4 arrives**, the OS may:

- Swap out **P2 to disk**
- Load **P4 into memory**

Advantages of Swapping

Advantage	Explanation
Better memory utilization	Memory used efficiently
Supports multitasking	Multiple processes can run
Improves CPU usage	Reduces idle time

Disadvantages of Swapping

Disadvantage	Explanation
High disk I/O	Frequent data transfer
Slower performance	Disk access slower than RAM
Swap time overhead	Time needed to move processes

Conditions Required for Swapping

Requirement	Explanation
Fast secondary storage	Disk must support quick access
Process relocation	Process must be movable in memory

Key Points for Exams

- Swapping moves processes between **RAM and disk**
- Two operations:
 - Swap Out
 - Swap In
- Helps manage **limited memory**
- Supports **multiprogramming systems**

Short Conclusion

Swapping is a memory management technique used to temporarily transfer processes between main memory and secondary storage. It helps the operating system manage limited memory resources and allows more processes to execute in a multitasking environment.

Demand Paging

1. Definition : Demand Paging is a memory management technique in which **pages are loaded into memory only when they are required during execution.**

Instead of loading the entire program into memory at once, the operating system loads **only the necessary pages**, which improves memory efficiency.

Simple exam definition : Demand paging is a technique in which pages are loaded into memory only when they are needed.

Basic Concept of Demand Paging

In demand paging:

1. The program is divided into **pages**.
2. Initially, only a few pages are loaded into memory.
3. When a process tries to access a page that is not in memory, the OS loads it from disk.

This process is called **page fault handling**.

Page Fault

A **Page Fault** occurs when a process tries to access a page that is **not currently present in main memory**.

Steps during a page fault:

1. The CPU checks the page table.
2. If the page is not in memory, a **page fault interrupt** occurs.
3. The operating system loads the required page from disk.
4. The page table is updated.
5. The process continues execution.

Demand Paging Diagram

Process Requests Page



Check Page Table



Page Present? ---- No ----> Page Fault



Load Page from Disk



Update Page Table



Continue Execution

Advantages of Demand Paging

Advantage	Explanation
Efficient memory use	Only required pages loaded
Faster program start	Entire program not loaded
Supports large programs	Programs larger than RAM can run

Disadvantages of Demand Paging

Disadvantage	Explanation
Page fault overhead	Disk access is slow
Performance reduction	Too many faults reduce speed
Complex implementation	Requires page tables and fault handling

Example

Suppose a program has **10 pages**, but only **3 pages are needed initially**.

Memory loads:

Page	Status
Page 1	Loaded
Page 2	Loaded
Page 3	Loaded

This reduces memory usage.

Memory Protection and Security

1. Definition : Memory Protection is a mechanism used by the operating system to **prevent a process from accessing memory that does not belong to it.**

It ensures that processes operate **within their assigned memory area.**

Simple exam definition : Memory protection is a technique that prevents processes from accessing unauthorized memory locations.

Purpose of Memory Protection

Purpose	Explanation
Prevent illegal access	Protect system memory
Improve system stability	Avoid program crashes
Protect data	Prevent unauthorized modification

How Memory Protection Works

The operating system uses hardware mechanisms such as:

Mechanism	Purpose
Base register	Starting address of process memory
Limit register	Maximum memory size allowed
Memory protection bits	Control read/write access

Example

Suppose a process has memory range:

Base Address = 1000

Limit = 500

Valid memory addresses:

1000 → 1499

If the process tries to access address **1500**, the OS **blocks the access.**

Memory Protection Diagram

Process Memory Range

1000 → Start Address

1499 → End Address

Access Outside Range → Not Allowed

Security Benefits of Memory Protection

Benefit	Explanation
Prevents data corruption	Processes cannot overwrite others
Protects operating system	User programs cannot modify OS memory
Improves system reliability	Prevents crashes

Key Points for Exams

- Demand paging loads **pages only when needed**
- Page fault occurs when page is **not in memory**
- Memory protection prevents **unauthorized memory access**
- Uses **base register and limit register**

Short Conclusion

Demand paging improves memory efficiency by loading pages only when they are required, reducing memory usage and supporting larger programs. Memory protection ensures system security by preventing processes from accessing unauthorized memory locations, thereby maintaining system stability and reliability.

Segmentation

1. Introduction: Segmentation is a memory management technique in which a program is divided into variable-sized logical parts called:

Segments

Each segment represents a logical unit of the program such as:

- code
- functions
- arrays
- stack
- data

Unlike paging:

- segmentation is based on logical structure of program

Definition: Segmentation is a memory management scheme that divides a process into variable-sized logical segments.

Simple Exam Definition: Segmentation is a memory management technique where programs are divided into logical variable-sized segments.

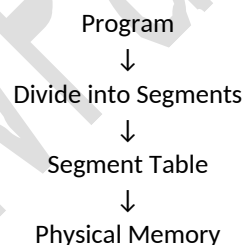
Why Segmentation is Needed

Programs naturally contain different sections:

- main program
- procedures
- stack
- data section

Segmentation organizes memory according to program structure.

Basic Idea



Example of Segments

Segment Number	Segment Type
0	Main Program
1	Function
2	Stack
3	Array/Data

Segment Structure

Each segment has:

- Segment Number
- Offset

Logical address format:

<Segment Number, Offset>

Example

<2, 150>

Meaning:

- Segment number = 2
- Offset inside segment = 150

Segment Table

OS maintains a Segment Table for each process.

It stores:

- base address
- segment limit

Segment Table Entry

Field	Meaning
Base	Starting physical address
Limit	Segment size

Segment Address Translation

Logical address:

<segment number, offset>

MMU checks:

- valid segment
- offset within limit

Then computes:

$$\text{Physical address} = \text{Base} + \text{Offset}$$

Segmentation Diagram

Logical Address

<Segment, Offset>



Segment Table



Base + Offset



Physical Address

Example of Address Translation

Suppose:

Segment	Base	Limit
0	1000	500
1	3000	400

Logical address:

<1, 200>

Calculation:

$$3000 + 200 = 3200$$

Physical address:

3200

Characteristics of Segmentation

Characteristic	Explanation
Variable-sized divisions	Segments differ in size
Logical division	Based on program structure
Supports protection	Segment-level security
Independent segments	Easy sharing

Advantages of Segmentation

Advantage	Explanation
Logical organization	Matches program structure
Easy sharing	Share individual segments
Better protection	Segment-wise control
Modular programming	Separate logical units

Disadvantages of Segmentation

Disadvantage	Explanation
External fragmentation	Free space scattered
Complex memory allocation	Variable sizes difficult
Overhead	Segment table maintenance

Segmentation vs Paging

Feature	Segmentation	Paging
Division type	Logical	Physical
Size	Variable	Fixed
Fragmentation	External	Internal
User visibility	Visible	Transparent

Real-Life Analogy

Book Example

Book Part	Segment
Chapter 1	Segment 1
Chapter 2	Segment 2
Appendix	Segment 3

Each chapter acts like a separate segment.

Applications of Segmentation

Application	Usage
Modular programming	Separate modules
Dynamic linking	Shared libraries
Memory protection	Secure execution
Multitasking systems	Organized memory

Important Exam Points

- Segmentation divides program logically
- Segments are variable-sized
- Logical address:
<segment number, offset>
- Segment table stores base and limit
- Suffers from external fragmentation

Key Terms

Term	Meaning
Segment	Logical program unit
Segment Table	Stores segment information
Base Address	Starting physical address
Limit	Segment size
Offset	Position inside segment

Short Conclusion

Segmentation is a memory management technique that divides programs into logical variable-sized segments such as code, stack, and data. Each segment has its own base address and limit stored in the segment table. Segmentation supports logical organization, modular programming, and memory protection, but may suffer from external fragmentation.

Segmentation with Paging

1. Introduction: Segmentation with Paging is a combined memory management technique that uses:

- segmentation
AND
- paging together

It combines advantages of both methods:

- logical division from segmentation
- efficient memory allocation from paging

Definition: Segmentation with Paging is a memory management scheme in which each segment is further divided into pages.

Simple Exam Definition: Segmentation with paging combines segmentation and paging to improve memory management efficiency.

Why Combination is Needed

Segmentation alone:

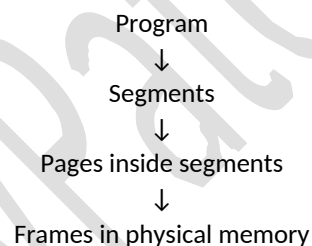
- suffers from external fragmentation

Paging alone:

- lacks logical program structure

Combining both solves these problems.

Basic Idea



Working Principle

Step 1

Program divided into:
Segments

Step 2

Each segment divided into:
Pages

Step 3

Pages loaded into:
Frames
in physical memory.

Logical Address Format

Logical address contains:

<Segment Number, Page Number, Offset>

Components

Component	Meaning
Segment Number	Select segment
Page Number	Select page inside segment
Offset	Exact location inside page

Address Translation Process

Logical Address

<Segment, Page, Offset>



Segment Table



Page Table



Frame Number



Physical Address

Segment Table

Stores:

- page table address
- segment limit

Page Table

Stores:

- frame number for each page

Example

Suppose:

Logical address:

<2, 3, 100>

Meaning:

- Segment = 2
- Page = 3
- Offset = 100

Address Translation Steps

Step 1

Use segment number to find page table.

Step 2

Use page number to find frame.

Step 3

Combine frame number with offset.

Final Physical Address

Frame Number + Offset

Segmentation with Paging Diagram

Logical Address

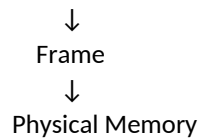
<Seg, Page, Offset>



Segment Table



Page Table



Characteristics

Characteristic	Explanation
Hybrid technique	Combines two methods
Logical organization	From segmentation
Efficient allocation	From paging
Reduced fragmentation	Better memory use

Advantages

Advantage	Explanation
Reduces external fragmentation	Paging used
Logical program structure	Segmentation support
Better memory utilization	Efficient allocation
Improved protection	Segment-based control

Disadvantages

Disadvantage	Explanation
Complex implementation	Two-level mapping
Extra overhead	Segment + page tables
Slower translation	Multiple lookups

Segmentation vs Segmentation with Paging

Feature	Segmentation	Segmentation with Paging
Fragmentation	External	Reduced
Structure	Segments only	Segments + pages
Complexity	Lower	Higher
Memory efficiency	Moderate	Better

Real-Life Analogy

Library Example

Library Concept	OS Concept
Section	Segment
Books inside section	Pages
Shelf location	Frames

Applications

Application	Usage
Modern operating systems	Advanced memory management
Virtual memory systems	Efficient allocation
Multitasking systems	Organized execution

Important Exam Points

- Combination of segmentation and paging
- Each segment divided into pages
- Logical address:
<segment, page, offset>
- Reduces external fragmentation
- Uses both segment table and page table

Key Terms

Term	Meaning
Segment	Logical division
Page	Fixed-size block
Frame	Physical memory block
Segment Table	Segment information
Page Table	Page mapping

Short Conclusion

Segmentation with Paging is a hybrid memory management technique that combines the logical structure of segmentation with the efficient allocation of paging. Programs are divided into segments, and each segment is further divided into pages. This method reduces external fragmentation while maintaining logical organization and efficient memory utilization.

Virtual Memory Concept

1. Introduction: Modern programs are very large and may require more memory than physically available in RAM.

To solve this problem, operating systems use: Virtual Memory

Virtual memory allows execution of large programs even if complete program is not loaded into main memory. It creates an illusion that:

Very large memory is available

Definition: Virtual Memory is a memory management technique that allows execution of processes larger than the available physical memory.

Simple Exam Definition: Virtual memory is a technique that uses secondary storage as an extension of main memory.

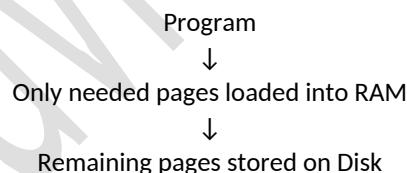
Basic Idea

Only required parts of a program are loaded into RAM.

Remaining parts stay in:

Secondary Storage (Disk)

Virtual Memory Flow



Why Virtual Memory is Needed

Programs today are:

- large
- complex
- memory-intensive

Physical RAM is limited.

Virtual memory solves this limitation.

Main Goals of Virtual Memory

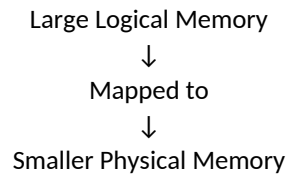
Goal	Explanation
Execute large programs	Beyond RAM size
Better memory utilization	Load only needed pages
Support multiprogramming	More processes in memory
Improve CPU utilization	More active processes

Working Principle

Virtual memory works using:

- paging
- demand paging
- address translation

Core Idea



Virtual Address Space

A process sees:

Large virtual address space
even if actual RAM is smaller.

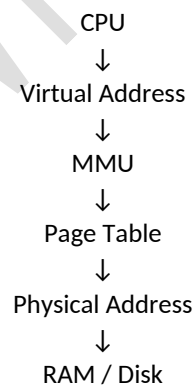
Physical Memory vs Virtual Memory

Feature	Physical Memory	Virtual Memory
Actual RAM	Yes	No
Storage location	Main memory	Disk + RAM
Size	Limited	Larger
Speed	Faster	Slower than RAM

Virtual Memory Components

Component	Purpose
RAM	Stores active pages
Disk	Stores remaining pages
MMU	Address translation
Page Table	Maps pages to frames

Virtual Memory Diagram



Address Translation in Virtual Memory

CPU generates:

Logical (Virtual) Address

MMU converts it into:

Physical Address

using page tables.

Example

Suppose:

- Program size = 10 GB
- RAM size = 4 GB

Without virtual memory:

Program cannot execute

With virtual memory:

- only needed pages loaded
- remaining pages stay on disk

Program executes successfully.

Demand Paging in Virtual Memory

Virtual memory commonly uses:

Demand Paging

Pages loaded:

- only when needed

This improves efficiency.

Advantages of Virtual Memory

Advantage	Explanation
Large program execution	Beyond RAM size
Better memory utilization	Efficient loading
Increased multiprogramming	More processes
Process isolation	Better security
Efficient CPU usage	More active processes

Disadvantages of Virtual Memory

Disadvantage	Explanation
Slower than RAM	Disk access slower
Page fault overhead	Extra processing
Complex implementation	Requires hardware support
Thrashing possible	Excessive page faults

Characteristics of Virtual Memory

Characteristic	Explanation
Uses secondary storage	Disk acts as extension
Logical memory larger than physical	Illusion of huge memory
Dynamic loading	Pages loaded on demand
Supports multitasking	Many processes together

Virtual Memory vs Physical Memory

Feature	Virtual Memory	Physical Memory
Nature	Logical	Actual
Size	Very large	Limited
Storage	Disk + RAM	RAM only
Cost	Lower	Higher

Virtual Memory vs Cache Memory

Feature	Virtual Memory	Cache Memory
Purpose	Increase memory size	Increase speed
Storage medium	Disk	High-speed memory
Speed	Slower	Faster

Real-Life Analogy

Library Example

Suppose:

- Student uses only one chapter at a time
- Entire book stays in library

Similarly:

- only required pages stay in RAM
- remaining pages stay on disk

Applications of Virtual Memory

Application	Usage
Multitasking systems	Multiple processes
Large software applications	Huge programs
Database systems	Large datasets
Modern operating systems	Efficient memory use

Important Exam Points

- Virtual memory uses disk as extension of RAM
- Allows execution of programs larger than RAM
- Uses paging and demand paging
- MMU performs address translation
- Improves multiprogramming

Key Terms

Term	Meaning
Virtual Memory	Logical memory extension
Page	Fixed-size memory block
Frame	Physical memory block
MMU	Address translation hardware
Demand Paging	Load page only when needed

Important Formula/Concept

Virtual memory mapping:

Virtual address → *physical address*

Short Conclusion

Virtual Memory is a memory management technique that allows processes larger than physical memory to execute efficiently by using secondary storage as an extension of RAM. Only required pages are loaded into memory, while remaining pages stay on disk. Virtual memory improves memory utilization, multiprogramming, and system performance in modern operating systems.

Page Replacement

1. Introduction: In virtual memory systems, processes are divided into pages.

When a required page is not available in memory:

- a page fault occurs
- operating system loads the required page into RAM

But if memory is already full:

OS must remove an existing page

This process is called:

Page Replacement

Definition: Page Replacement is the process of removing a page from memory and loading a new required page into memory.

Simple Exam Definition: Page replacement is a memory management technique used to replace pages when physical memory becomes full.

Why Page Replacement is Needed

Main memory has:

Limited number of frames

When all frames are occupied:

- new pages cannot be loaded directly

Therefore:

- one existing page must be removed

Basic Idea

Required Page Not Present



Page Fault Occurs



Memory Full ?



Yes



No

Replace Page Load Directly

Working Principle

Step 1

CPU requests a page.

Step 2

If page already in memory:

Normal execution continues

Step 3

If page absent:

Page Fault occurs

Step 4

If free frame exists:

- page loaded directly

Else:

- page replacement required

Step 5

OS selects a victim page.

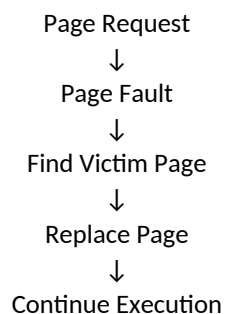
Step 6

Victim page removed.

Step 7

Required page loaded into free frame.

Page Replacement Flow



Victim Page

The page selected for removal is called:
Victim Page

Goals of Page Replacement

Goal	Explanation
Reduce page faults	Improve performance
Efficient memory usage	Better frame utilization
Faster execution	Less disk access

Need for Efficient Algorithms

Poor page replacement:

- increases page faults
- slows system performance

Good algorithms:

- minimize page faults

Types of Page Replacement Algorithms

Algorithm	Idea
FIFO	Remove oldest page
Optimal	Remove page used farthest in future
LRU	Remove least recently used page

1. FIFO (First In First Out)

Oldest loaded page removed first.

Example

Pages entered:

1 → 2 → 3

If replacement needed:

Page 1 removed first

2. Optimal Page Replacement

Replace page that:

will not be used for longest future time

Produces minimum page faults.

Limitation

Future knowledge required.

Not practical in real systems.

3. LRU (Least Recently Used)

Replace page that:

has not been used for longest past time

Based on recent usage history.

Page Replacement Example

Suppose:

Frames: 3 Frames

Pages in memory: 1 2 3

New page requested: 4

Memory full.

OS removes one page and loads page 4.

Characteristics of Page Replacement

Characteristic	Explanation
Used in virtual memory	Demand paging systems
Triggered by page fault	Memory full condition
Uses replacement algorithms	Victim selection
Improves memory efficiency	Better frame usage

Advantages of Page Replacement

Advantage	Explanation
Efficient memory usage	Limited RAM utilization
Supports virtual memory	Large program execution
Better multiprogramming	More active processes

Disadvantages of Page Replacement

Disadvantage	Explanation
Disk overhead	Swapping pages costly
Performance reduction	Frequent faults slow system
Complex algorithms	Extra management needed

Page Replacement vs Swapping

Feature	Page Replacement	Swapping
Unit moved	Individual page	Entire process
Efficiency	Higher	Lower
Memory usage	Fine-grained	Large-scale

Page Replacement vs Demand Paging

Feature	Demand Paging	Page Replacement
Purpose	Load pages on demand	Remove pages when full
Trigger	Page request	Memory full

Belady's Anomaly

Some algorithms like FIFO may show:

More frames → More page faults

This unusual situation is called:

Belady's Anomaly

Real-Life Analogy

Parking Area Example

Suppose:

- parking full
- new car arrives

One existing car must leave for new car to enter.

This is similar to page replacement.

Applications of Page Replacement

Application	Usage
Virtual memory systems	Efficient memory handling
Multitasking systems	Process management
Modern operating systems	Dynamic memory allocation

Important Exam Points

- Page replacement occurs when memory is full
- Triggered by page faults

- Victim page selected for removal
- FIFO, LRU, Optimal are important algorithms
- Goal is minimizing page faults

Key Terms

Term	Meaning
Page Fault	Requested page absent
Victim Page	Page selected for removal
Frame	Physical memory block
FIFO	First In First Out
LRU	Least Recently Used

Important Concept

Page replacement occurs when:

Requested page $\notin$ Memory

AND

No free frame available

Short Conclusion

Page Replacement is a memory management technique used in virtual memory systems when physical memory becomes full. The operating system removes an existing page and loads the required page into memory. Efficient page replacement algorithms such as FIFO, LRU, and Optimal help reduce page faults and improve overall system performance.

FIFO Page Replacement Algorithm

1. Introduction

FIFO stands for: First In First Out

FIFO Page Replacement is one of the simplest page replacement algorithms used in operating systems.

The basic idea is: The page that entered memory first is removed first.

It works like a queue.

Definition: FIFO Page Replacement is a page replacement algorithm in which the oldest loaded page is replaced first.

Simple Exam Definition: FIFO replaces the page that has been in memory for the longest time.

Basic Idea

Pages are stored in memory in arrival order.

When memory becomes full:

- oldest page removed
- new page inserted

FIFO Queue Concept

Oldest Page → Removed First

Newest Page → Added Last

Working Principle

Step 1

Pages enter memory one by one.

Step 2

If free frame available:

- page loaded directly

Step 3

If memory full:

- remove oldest page
- load new page

FIFO Flow

Page Request



Page in Memory ?



Yes



No

Hit

Page Fault



Free Frame ?



Yes



No

Load

Remove Oldest Page

Example Problem

Given

Reference String:

1 2 3 4 1 2 5 1 2 3 4 5

Frames:

3 Frames

Step-by-Step FIFO Solution

Step	Page	Frame Contents	Page Fault
1	1	1	Yes
2	2	1 2	Yes
3	3	1 2 3	Yes
4	4	2 3 4	Yes
5	1	3 4 1	Yes
6	2	4 1 2	Yes
7	5	1 2 5	Yes
8	1	1 2 5	No
9	2	1 2 5	No
10	3	2 5 3	Yes
11	4	5 3 4	Yes
12	5	5 3 4	No

Total Page Faults

Total\ Page\ Faults = 9

FIFO Diagram Concept

Queue Order:

Front → Oldest Page

Rear → Newest Page

FIFO Queue Example

Suppose frames contain:

1 2 3

Page 4 arrives.

FIFO removes: 1

because it entered first.

New frames:

2 3 4

Characteristics of FIFO

Characteristic	Explanation
Queue-based	Oldest removed first
Simple algorithm	Easy implementation
No usage tracking	Ignores page usage
Old pages removed	Based on arrival time

Advantages of FIFO

Advantage	Explanation
Very simple	Easy understanding
Low overhead	Minimal tracking
Easy implementation	Queue structure

Disadvantages of FIFO

Disadvantage	Explanation
Poor performance	Frequently used pages removed
Ignores page importance	Only age considered
Belady's anomaly possible	More frames may increase faults

Belady's Anomaly

FIFO may show:

More Frames → More Page Faults

This unusual condition is called:

Belady's Anomaly

Example of Belady's Anomaly

Sometimes:

- 3 frames produce fewer faults
- 4 frames produce more faults

This occurs mainly in FIFO.

FIFO vs LRU

Feature	FIFO	LRU
Basis	Arrival time	Recent usage
Complexity	Simple	More complex
Performance	Lower	Better

FIFO vs Optimal

Feature	FIFO	Optimal
Future knowledge needed	No	Yes
Performance	Moderate	Best possible
Practicality	Practical	Theoretical

Real-Life Analogy

Queue in Ticket Counter

First person entering queue:

leaves first

Same principle used in FIFO page replacement.

Applications of FIFO

Application	Usage
Basic OS memory management	Simple systems
Queue scheduling systems	Ordered processing
Educational understanding	Learning algorithms

Important Exam Points

- FIFO means First In First Out
- Oldest page replaced first
- Uses queue structure
- Simple implementation
- Belady's anomaly possible

Key Terms

Term	Meaning
FIFO	First In First Out
Page Fault	Missing page in memory
Victim Page	Page removed
Queue	Ordered structure

Important Formula/Concept

FIFO replacement rule:

Replace oldest loaded page

Short Conclusion

FIFO Page Replacement is a simple memory management algorithm in which the oldest page in memory is replaced first when memory becomes full. It uses a queue-based structure and is easy to implement, but it may produce higher page faults and can suffer from Belady's anomaly.

LRU Page Replacement Algorithm

1. Introduction

LRU stands for: Least Recently Used

LRU Page Replacement removes the page that has not been used for the longest period of time.

The basic assumption is: Pages used recently will probably be used again soon.

Definition: LRU Page Replacement is an algorithm that replaces the page which has not been used for the longest time.

Simple Exam Definition: LRU replaces the least recently used page from memory.

Basic Idea

When memory becomes full:

- OS checks page usage history
- removes oldest unused page

Working Principle

Step 1

Page requested by CPU.

Step 2

If page already exists:

Page Hit

Update recent usage.

Step 3

If page absent:

Page Fault

Step 4

If memory full:

- remove least recently used page
- load new page

LRU Flow

Page Request



Page Present ?



Yes

Hit



No

Page Fault



Replace Least Recently Used Page

Example Problem

Given

Reference String:

7 0 1 2 0 3 0 4 2 3 0 3 2

Frames:

3 Frames

Step-by-Step LRU Solution

Step	Page	Frame Contents	Page Fault
1	7	7	Yes
2	0	7 0	Yes
3	1	7 0 1	Yes
4	2	0 1 2	Yes
5	0	0 1 2	No
6	3	0 2 3	Yes
7	0	0 2 3	No
8	4	0 3 4	Yes
9	2	0 4 2	Yes
10	3	4 2 3	Yes
11	0	2 3 0	Yes
12	3	2 3 0	No
13	2	2 3 0	No

Total Page Faults

Total \ Page \ Faults = 9

Example Explanation

Suppose frames contain:

1 2 3

Recent usage:

- 1 used long ago
- 2 used recently
- 3 used recently

New page arrives: 4

LRU removes: 1
because it was least recently used.

Characteristics of LRU

Characteristic	Explanation
Usage-based	Uses recent history
Better performance	Fewer faults
Dynamic replacement	Based on activity
Practical algorithm	Commonly used

Advantages of LRU

Advantage	Explanation
Better performance	Lower page faults
Uses locality principle	Recent pages reused
No Belady's anomaly	Stable behavior

Disadvantages of LRU

Disadvantage	Explanation
Complex implementation	Usage tracking needed
Higher overhead	Time/history maintenance
Extra hardware/software	Counters or stacks needed

LRU Implementation Methods

Method	Explanation
Counter Method	Time stamp tracking
Stack Method	Maintain usage order

LRU vs FIFO

Feature	LRU	FIFO
Basis	Recent usage	Arrival time
Performance	Better	Lower
Belady's anomaly	No	Yes
Complexity	Higher	Lower

LRU vs Optimal

Feature	LRU	Optimal
Uses past history	Yes	No
Uses future knowledge	No	Yes
Practical	Yes	No

Real-Life Analogy

Recently Used Books

Suppose:

- books not used for long time removed from desk first

Similar to LRU replacement.

Applications of LRU

Application	Usage
Operating systems	Virtual memory
CPU cache	Cache replacement
Database systems	Buffer management

Important Exam Points

- LRU means Least Recently Used
- Removes least recently used page
- Uses usage history
- Better than FIFO
- No Belady's anomaly

Key Terms

Term	Meaning
LRU	Least Recently Used
Page Fault	Missing page
Locality Principle	Recently used pages reused again

Important Concept

LRU replacement rule:

Replace Least Recently Used Page

Short Conclusion

LRU Page Replacement is an efficient memory management algorithm that replaces the page which has not been used for the longest time. It improves performance by reducing page faults and uses the principle of locality, although it requires additional tracking overhead.

Optimal Page Replacement Algorithm

1. Introduction

Optimal Page Replacement is the best theoretical page replacement algorithm.

It replaces the page that:

will not be used for the longest future time.

This algorithm produces:

Minimum possible page faults

Definition: Optimal Page Replacement is an algorithm that replaces the page whose next use will occur farthest in the future.

Simple Exam Definition: Optimal page replacement removes the page that will not be used for the longest future period.

Basic Idea

When replacement required:

- predict future page references
- remove page needed farthest in future

Why Called Optimal?

Because: No other algorithm can produce fewer page faults.

Working Principle

Step 1

Page requested.

Step 2

If page present:

Page Hit

Step 3

If page absent:

Page Fault

Step 4

If memory full:

- check future references
- replace page used farthest in future

Optimal Flow

Page Fault



Check Future Usage



Replace Page Used Farthest in Future

Example Problem

Given

Reference String:

7 0 1 2 0 3 0 4 2 3 0 3 2

Frames:

3 Frames

Step-by-Step Optimal Solution

Step	Page	Frame Contents	Page Fault
1	7	7	Yes
2	0	7 0	Yes
3	1	7 0 1	Yes
4	2	2 0 1	Yes
5	0	2 0 1	No
6	3	2 0 3	Yes
7	0	2 0 3	No
8	4	2 4 3	Yes
9	2	2 4 3	No
10	3	2 4 3	No
11	0	2 0 3	Yes
12	3	2 0 3	No
13	2	2 0 3	No

Total Page Faults

Total Page Faults = 7

Example Explanation

Suppose frames contain:

1 2 3

Future usage:

- 1 used very late
- 2 used soon
- 3 used soon

New page arrives: 4

Optimal removes: 1

because it is needed farthest in future.

Characteristics of Optimal Algorithm

Characteristic	Explanation
Future-based	Uses future references
Minimum faults	Best possible performance
Theoretical algorithm	Not practical

Benchmark algorithm	Used for comparison
----------------------------	---------------------

Advantages of Optimal Algorithm

Advantage	Explanation
Lowest page faults	Best performance
Ideal benchmark	Compare other algorithms
No Belady's anomaly	Stable behavior

Disadvantages of Optimal Algorithm

Disadvantage	Explanation
Future knowledge required	Impossible in practice
Not implementable directly	Theoretical only

Optimal vs FIFO

Feature	Optimal	FIFO
Basis	Future usage	Arrival order
Page faults	Lowest	Higher
Practicality	Theoretical	Practical

Optimal vs LRU

Feature	Optimal	LRU
Future knowledge	Required	Not required
Performance	Best	Near-optimal
Practicality	Low	High

Real-Life Analogy

Smart Parking System

Suppose:

- system predicts which car will leave last
- removes that parking reservation first

Similar to optimal replacement.

Applications of Optimal Algorithm

Application	Usage
Performance comparison	Benchmarking
Research	Memory management studies
OS education	Algorithm analysis

Important Exam Points

- Optimal algorithm gives minimum page faults
- Replaces page used farthest in future
- Theoretical algorithm
- Future knowledge required
- Used as benchmark

Key Terms

Term	Meaning
Optimal Replacement	Best page replacement
Future Reference	Upcoming page usage
Benchmark	Standard comparison

Important Concept

Optimal replacement rule:

Replace Page Used Farthest in future

Short Conclusion

Optimal Page Replacement is a theoretical page replacement algorithm that replaces the page whose next use occurs farthest in the future. It produces the minimum possible page faults and is commonly used as a benchmark for comparing other page replacement algorithms, although it cannot be practically implemented because future page references are unknown.

Frame Allocation

1. Introduction

In paging systems, physical memory is divided into fixed-size blocks called: Frames

Processes are divided into: Pages

Operating system must decide: How many frames should be given to each process

This process is called: Frame Allocation

Definition: Frame Allocation is the process of assigning available memory frames to different processes.

Simple Exam Definition: Frame allocation is a memory management technique used to distribute physical memory frames among processes.

Why Frame Allocation is Needed

Main memory has:

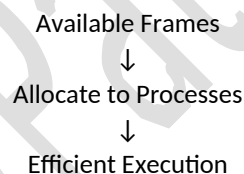
Limited number of frames

Many processes compete for these frames.

OS must allocate frames efficiently to:

- reduce page faults
- improve performance
- support multiprogramming

Basic Idea



Goals of Frame Allocation

Goal	Explanation
Reduce page faults	Better performance
Efficient memory usage	Proper distribution
Fair allocation	All processes get memory
Improve CPU utilization	Fewer waiting processes

Minimum Number of Frames

Every process requires:

Minimum frames for execution

Without minimum frames:

- process cannot execute correctly

Example

Suppose instruction requires:

- 2 operands
- 1 instruction page

Then process may require:

Minimum 3 frames

Types of Frame Allocation

Method	Description
Equal Allocation	Equal frames to all processes
Proportional Allocation	Frames based on process size
Priority Allocation	Frames based on priority

1. Equal Frame Allocation

Concept

All processes receive: Equal number of frames

Example

Suppose:

- Total frames = 12
- Processes = 3

Then:

Frames\ Per\ Process = $\frac{12}{3} = 4$

Each process gets:

4 Frames

Advantages

Advantage	Explanation
Simple implementation	Easy calculation
Fair distribution	Equal allocation

Disadvantages

Disadvantage	Explanation
Ignores process size	Small and large processes treated same
Possible inefficiency	Unequal memory requirements

2. Proportional Allocation

Concept

Frames allocated according to: Process size

Larger processes get more frames.

Formula

$$Frames_i = \frac{Size_i}{Total\ size} \times Total\ frames$$

Example

Suppose:

Process	Size
P1	20 KB
P2	40 KB

Total frames: 30

P2 receives more frames than P1.

Advantages

Advantage	Explanation
Better efficiency	Based on need
Reduced page faults	Large processes supported

Disadvantages

Disadvantage	Explanation
Slightly complex	Size calculation needed

Unequal allocation	Small processes get fewer frames
---------------------------	----------------------------------

3. Priority Allocation

Concept

Frames allocated according to: Process priority

High-priority processes receive more frames.

Example

Process	Priority
P1	High
P2	Low

P1 receives: More Frames

Advantages

Advantage	Explanation
Better important-process performance	Priority support
Efficient critical execution	Important tasks favored

Disadvantages

Disadvantage	Explanation
Starvation possible	Low-priority processes suffer
Unfair allocation	Priority imbalance

Local vs Global Allocation

Local Allocation

Process can replace:

Only its own frames

Global Allocation

Process may replace:

Frames of other processes

Comparison

Feature	Local Allocation	Global Allocation
Replacement scope	Own frames only	Any frame
Isolation	Better	Lower
Flexibility	Lower	Higher

Characteristics of Frame Allocation

Characteristic	Explanation
Dynamic allocation	Frames change over time
Memory management support	Paging systems
Performance impact	Affects page faults
Resource sharing	Multiple processes

Importance of Frame Allocation

Importance	Explanation
Efficient paging	Better replacement
Reduced page faults	Improved speed
Fair memory usage	Balanced execution
Multiprogramming support	Many active processes

Advantages of Proper Frame Allocation

Advantage	Explanation
Better system performance	Fewer faults
Efficient RAM utilization	Balanced memory
Faster execution	Less swapping

Disadvantages of Poor Allocation

Disadvantage	Explanation
Increased page faults	Slower system
Thrashing possible	Excessive swapping
Poor performance	CPU waiting

Real-Life Analogy

Hostel Room Allocation

Suppose:

- rooms distributed among students
- larger groups need more rooms

Similar to proportional frame allocation.

Applications of Frame Allocation

Application	Usage
Virtual memory systems	Paging support
Multitasking OS	Process memory management
Demand paging	Frame distribution

Important Exam Points

- Frame allocation distributes memory frames
- Equal allocation gives same frames
- Proportional allocation depends on process size
- Priority allocation depends on process priority
- Poor allocation may cause thrashing

Key Terms

Term	Meaning
Frame	Physical memory block
Page	Logical memory block
Equal Allocation	Same frames to all
Proportional Allocation	Based on size
Priority Allocation	Based on priority

Important Formula

Proportional allocation formula:

$$Frames_i = \frac{Size_i}{Total\ size} \times Total\ frames$$

Short Conclusion

Frame Allocation is a memory management technique used in paging systems to distribute physical memory frames among processes. Operating systems use methods such as equal allocation, proportional allocation, and priority allocation to improve memory utilization, reduce page faults, and support efficient multitasking.

Thrashing

1. Introduction

In virtual memory systems, processes continuously generate page requests.

If page faults become excessively high:

- CPU spends most of its time swapping pages

- very little actual execution occurs

This condition is called:

Thrashing

Thrashing severely reduces system performance.

Definition: Thrashing is a condition in which the operating system spends more time handling page faults than executing processes.

Simple Exam Definition: Thrashing is excessive paging activity caused by insufficient memory frames.

Basic Idea

Too Many Page Faults



Continuous Page Swapping



CPU Utilization Decreases



Thrashing Occurs

Why Thrashing Occurs

Thrashing mainly occurs because:

Processes do not have enough frames

As a result:

- pages frequently removed
- same pages repeatedly reloaded

Main Causes of Thrashing

Cause	Explanation
Insufficient frames	Not enough memory
High degree of multiprogramming	Too many processes
Excessive page faults	Frequent page loading
Poor frame allocation	Improper distribution

Thrashing Process Flow

Process Requests Page



Page Fault



Replace Existing Page



Another Needed Page Removed



New Page Fault



Continuous Repetition

Example of Thrashing

Suppose:

- Process requires 10 pages
- Only 3 frames allocated

Then:

- pages constantly replaced
- page faults occur repeatedly

This creates thrashing.

Symptoms of Thrashing

Symptom	Explanation
High page fault rate	Continuous faults
Low CPU utilization	CPU waiting for disk
Heavy disk activity	Excessive swapping
Slow system performance	Reduced execution speed

Thrashing Diagram

Low Frames



More Page Faults



More Swapping



Lower CPU Utilization



Thrashing

Relationship Between CPU Utilization and Thrashing

Initially:

- multiprogramming increases CPU utilization

But after a limit:

- page faults increase
- CPU utilization drops

CPU Utilization Graph Concept

More Processes



More Paging



Thrashing



CPU Utilization Falls

Effects of Thrashing

Effect	Explanation
System slowdown	Reduced performance
High disk usage	Excessive swapping
CPU wastage	Idle waiting
Reduced throughput	Fewer completed tasks

Detection of Thrashing

Operating system monitors:

- page fault frequency
- CPU utilization
- memory usage

High page fault frequency indicates thrashing.

Methods to Control Thrashing

Method	Purpose
Reduce multiprogramming	Fewer processes
Increase frames	More memory
Better frame allocation	Proper distribution

Working Set Model	Locality-based allocation
--------------------------	---------------------------

Page Fault Frequency (PFF)

OS maintains:

- upper page fault limit
- lower page fault limit

Rule

Condition	Action
High page fault rate	Allocate more frames
Low page fault rate	Remove extra frames

Advantages of Thrashing Control

Advantage	Explanation
Better CPU utilization	More useful execution
Reduced page faults	Less swapping
Improved performance	Faster system

Disadvantages of Thrashing

Disadvantage	Explanation
Severe slowdown	Very poor performance
High disk overhead	Continuous I/O
Resource wastage	CPU idle time

Thrashing vs Swapping

Feature	Thrashing	Swapping
Nature	Problem condition	Memory technique
Cause	Excessive faults	Process movement
Performance	Poor	Controlled

Real-Life Analogy

Student Changing Books Repeatedly

Suppose:

- student continuously changes books
- very little actual study happens

This is similar to thrashing.

Characteristics of Thrashing

Characteristic	Explanation
Excessive paging	Continuous replacement
High page fault rate	Frequent faults
Low CPU utilization	CPU idle often
Heavy disk activity	Constant swapping

Applications of Thrashing Study

Application	Usage
Performance tuning	Optimize memory
OS memory management	Improve paging
Virtual memory systems	Better allocation

Important Exam Points

- Thrashing occurs due to excessive page faults
- Caused by insufficient frames

- CPU utilization decreases
- Heavy disk activity occurs
- Controlled using working set model and PFF

Key Terms

Term	Meaning
Thrashing	Excessive paging
Page Fault	Missing page
Multiprogramming	Multiple processes
Swapping	Page movement

Important Concept

Thrashing condition:

High Page Fault Rate righter row Thrashing

Short Conclusion

Thrashing is a serious performance problem in virtual memory systems where excessive page faults cause the operating system to spend most of its time swapping pages instead of executing processes. It occurs mainly due to insufficient memory frames and excessive multiprogramming. Proper frame allocation and working set management help control thrashing.

Working Set Model

1. Introduction

The Working Set Model is a technique used to control thrashing in virtual memory systems.

It is based on the idea: Processes actively use only a small set of pages at a time.

This set of actively used pages is called: Working Set

Definition: The Working Set Model is a memory management technique that keeps track of pages actively used by a process during a specific time interval.

Simple Exam Definition: Working set model is a method used to prevent thrashing by maintaining currently active pages in memory.

Basic Idea

Processes repeatedly access:

- recently used pages
- nearby pages

Keeping these pages in memory reduces page faults.

Working Set

A Working Set is:

The set of pages currently being used frequently by a process.

Working Set Window

Working set considers:

Recent page references

within a fixed interval called:

Window Size (Δ)

Example

Suppose recent page references are:

1 2 3 2 1 4

Working set may contain:

1 2 3 4

Working Set Model Principle

If:

- all active pages remain in memory

Then:

Page faults reduce
and thrashing decreases.

Working Set Diagram

Recent Page References



Working Set Determined



Keep Working Set in Memory



Reduce Thrashing

Locality of Reference

Working set model uses:

Locality Principle

Meaning:

- recently used pages likely reused again

Types of Locality

Type	Meaning
Temporal Locality	Recently used pages reused
Spatial Locality	Nearby pages accessed

Temporal Locality Example

Loop variables reused repeatedly

Spatial Locality Example

Sequential array access

Working Set Calculation

Working set depends on:

- recent references
- window size Δ

Working Set Formula Concept

Working set = Pages referenced in Last Δ references

Working Set Size

Situation	Effect
Small working set	Fewer frames needed
Large working set	More frames needed

Thrashing Control Using Working Set

OS checks:

- working set size
- available frames

If insufficient frames:

- reduce multiprogramming

Advantages of Working Set Model

Advantage	Explanation
Reduces thrashing	Active pages retained
Better performance	Fewer faults
Efficient frame allocation	Dynamic management
Improved CPU utilization	More execution

Disadvantages of Working Set Model

Disadvantage	Explanation
Complex implementation	Continuous monitoring
Extra overhead	Tracking references
Memory consumption	Additional management

Working Set vs Page Fault Frequency

Feature	Working Set Model	Page Fault Frequency
Basis	Active pages	Fault rate
Complexity	Higher	Lower
Accuracy	Better	Moderate

Characteristics of Working Set Model

Characteristic	Explanation
Dynamic memory management	Changes continuously
Locality-based	Uses recent activity
Thrashing prevention	Main purpose
Frame optimization	Efficient allocation

Real-Life Analogy

Study Table Example

Suppose:

- student keeps frequently used books on desk
- unused books removed

This is similar to working set management.

Applications of Working Set Model

Application	Usage
Virtual memory systems	Thrashing control
Modern operating systems	Dynamic allocation
Performance optimization	Efficient paging

Important Exam Points

- Working set contains active pages
- Uses locality principle
- Helps prevent thrashing
- Working set window represented by Δ
- Keeps frequently used pages in memory

Key Terms

Term	Meaning
Working Set	Active pages
Window Size (Δ)	Recent reference interval
Locality	Repeated nearby access
Temporal Locality	Recently used pages reused
Spatial Locality	Nearby pages accessed

Important Concept

Working set principle:

Keep Frequently Used Pages in Memory

Short Conclusion

The Working Set Model is a memory management technique used to reduce thrashing by keeping the actively used pages of a process in memory. It uses the principle of locality to determine which pages are frequently referenced. By maintaining the working set efficiently, operating systems reduce page faults and improve overall system performance.

Chirup Patel/16120